

GSAP MASTER NOTES (COMPLETE GUIDE)

This document contains detailed explanations of GSAP properties, methods, plugins, React integration, timelines, ScrollTrigger, animations, performance optimization, and advanced usage.

1. GSAP Introduction

GSAP (GreenSock Animation Platform) is a high-performance JavaScript animation library.

Why developers use GSAP:

- Extremely fast rendering engine
- Better browser compatibility
- Smooth animations
- Easy timeline management
- Works with React, Vue, Angular, SVG, Canvas, WebGL
- Advanced plugins for professional websites

Main use cases:

- Landing pages
- Portfolio websites
- Interactive storytelling
- Games
- Product showcases
- Smooth scroll websites

2. Installation

CDN:

```
<script src="https://cdn.jsdelivr.net/npm/gsap@3/dist/gsap.min.js"></script>
```

NPM:

```
npm install gsap
```

React:

```
npm install gsap @gsap/react
```

ScrollTrigger:

```
import { ScrollTrigger } from "gsap/ScrollTrigger";  
gsap.registerPlugin(ScrollTrigger);
```

3. gsap.to()

Definition:

Animates an element FROM current values TO target values.

Syntax:

```
gsap.to(target, vars)
```

Example:

```
gsap.to(".box", {  
  x: 300,  
  duration: 2  
});
```

Explanation:

- target = element selector
- vars = animation object

Properties:

- x → moves element horizontally
- y → moves element vertically
- duration → total animation time
- opacity → visibility
- rotation → rotate element
- scale → resize element

4. gsap.from()

Definition:

Starts animation FROM given values and ends at original state.

Example:

```
gsap.from(".box", {
  opacity: 0,
  y: -100,
  duration: 1
});
```

Explanation:

Element begins invisible and above the screen.
Then moves into original position.

5. gsap.fromTo()

Definition:

Defines both starting and ending animation states.

Syntax:

```
gsap.fromTo(
  target,
  fromVars,
  toVars
)
```

Example:

```
gsap.fromTo(
  ".box",
  { x: -200, opacity: 0 },
  { x: 0, opacity: 1, duration: 2 }
);
```

6. Important Position Properties

x:

Moves element horizontally using transform: translateX().

y:

Moves element vertically using transform: translateY().

xPercent:

Moves using percentage horizontally.

yPercent:

Moves using percentage vertically.

top/left:

Moves element using CSS positioning.
Avoid animating frequently because it causes reflow.

Best Practice:

Use x and y instead of top and left.

7. Rotation Properties

rotation:

Rotates element in 2D.

rotationX:

Rotates element on X-axis (3D effect).

rotationY:

Rotates element on Y-axis.

Example:

```
gsap.to(".box", {
  rotation: 360,
  duration: 2
});
```

8. Scale Properties

scale:
Scales both width and height.

scaleX:
Scales horizontally only.

scaleY:
Scales vertically only.

Example:
gsap.to(".box", {
 scale: 2,
 duration: 1
});

9. Opacity & Visibility

opacity:
Controls transparency.

0 = invisible
1 = fully visible

autoAlpha:
Controls opacity + visibility together.

Example:
gsap.to(".box", {
 autoAlpha: 0
});

Benefit:
display remains clean for interactions.

10. Timing Properties

duration:
Animation duration in seconds.

delay:
Wait time before animation starts.

repeat:
Number of repetitions.

repeat: -1
Means infinite loop.

repeatDelay:
Delay between repetitions.

yoyo:
Animation reverses after completion.

Example:
gsap.to(".box", {
 x: 300,
 repeat: -1,
 yoyo: true,
 duration: 1
});

11. Easing Properties

Definition:
Controls animation speed curve.

Common eases:

- linear
- power1
- power2
- power3
- power4
- bounce
- elastic
- expo
- circ
- back

Example:
ease: "bounce.out"

Meaning:
Animation bounces naturally.

12. Stagger

Definition:
Animates multiple elements one after another.

Example:
gsap.to(".box", {
 y: -100,
 stagger: 0.2
});

Advanced:
stagger: {
 each: 0.2,
 from: "center"
}

Properties:
• each → delay between items
• from → animation starting point
• grid → grid-based staggering

13. Timeline

Definition:
Sequences multiple animations.

Example:
const tl = gsap.timeline();

tl.to(".box1", { x: 100 })
 .to(".box2", { y: 100 })
 .to(".box3", { rotation: 360 });

Benefits:
• Better organization
• Smooth sequencing
• Easy control

14. Timeline Position Parameters

'+=1'
Starts after 1 second.

'-=1'
Starts 1 second earlier.

```
'<'
Starts with previous animation.

'>'
Starts after previous animation ends.

Example:
tl.to(".box1", { x: 100 })
  .to(".box2", { y: 100 }, "<");
```

15. Timeline Control Methods

```
play() → start animation
pause() → stop temporarily
reverse() → reverse animation
restart() → restart animation
seek() → jump to specific time
kill() → remove animation
```

```
Example:
tl.pause();
tl.play();
```

16. ScrollTrigger

Definition:
Triggers animations during scrolling.

Installation:
`import { ScrollTrigger } from "gsap/ScrollTrigger";`

Register:
`gsap.registerPlugin(ScrollTrigger);`

```
Example:
gsap.to(".box", {
  x: 400,
  scrollTrigger: {
    trigger: ".box",
    start: "top center",
    scrub: true
  }
});
```

17. ScrollTrigger Properties

trigger:
Element that activates scroll animation.

start:
Animation starting point.

end:
Animation ending point.

scrub:
Syncs animation with scroll.

pin:
Pins section during scrolling.

markers:
Shows debugging markers.

toggleActions:
Controls play/pause behavior.

snap:
Snaps animation sections.

anticipatePin:

Smooth pinning.

18. ScrollTrigger Start & End

Examples:

start: "top center"
Meaning:
Top of trigger reaches center viewport.

start: "top 80%"
Meaning:
Top hits 80% viewport height.

end: "+=500"
Meaning:
Animation lasts 500px scrolling.

19. React + GSAP

Basic Example:

```
import { useEffect, useRef } from "react";
import gsap from "gsap";

function App() {
  const box = useRef();

  useEffect(() => {
    gsap.to(box.current, {
      x: 200
    });
  }, []);

  return <div ref={box}></div>;
}
```

Why useRef?
Because React controls the DOM.

20. useGSAP Hook

Install:
npm install @gsap/react

Example:

```
useGSAP(() => {
  gsap.to(".box", {
    x: 300
  });
});
```

Benefits:

- Cleaner code
- Automatic cleanup
- Better React integration

21. SVG Animation

GSAP works perfectly with SVG.

Properties:

- strokeDasharray
- strokeDashoffset
- transformOrigin
- morphSVG

Example:

```
gsap.to("path", {
  strokeDashoffset: 0,
  duration: 2
});
```

22. Text Animation

GSAP can animate:

- letters
- words
- lines

Libraries:

- SplitText
- SplitType

Example:

```
gsap.from(".title span", {
  opacity: 0,
  y: 50,
  stagger: 0.05
});
```

23. Draggable Plugin

Definition:

Makes elements draggable.

Example:

```
Draggable.create(".box", {
  type: "x,y"
});
```

Options:

- bounds
- inertia
- edgeResistance

24. Flip Plugin

Definition:

Animates layout transitions.

FLIP:

- First
- Last
- Invert
- Play

Example:

```
Flip.from(state, {
  duration: 1
});
```

25. MotionPath Plugin

Definition:

Moves elements along SVG paths.

Example:

```
gsap.to(".ball", {
  motionPath: {
    path: "#path"
  }
});
```

26. GSAP with Three.js

GSAP can animate 3D objects.

Example:
`gsap.to(mesh.rotation, {
 y: Math.PI * 2,
 duration: 5
});`

27. Performance Optimization

Best Practices:

- Use transform instead of width/top/left
- Avoid layout thrashing
- Use timelines
- Minimize repaint
- Animate opacity & transforms only
- Use will-change carefully

28. Common Mistakes

- Forgetting plugin registration
- Wrong selectors
- Missing cleanup in React
- Using top/left too much
- Overusing heavy animations

29. Advanced Features

GSAP supports:

- keyframes
- matchMedia
- context()
- quickSetter()
- Observer plugin
- ScrollSmoother
- MorphSVG
- SplitText

30. GSAP Interview Questions

1. Difference between `gsap.to` and `gsap.from`?
2. What is `ScrollTrigger`?
3. Why are transforms faster?
4. Explain timelines.
5. What is scrub?
6. Explain stagger.
7. Why use `useRef` in React?
8. Difference between ease types?
9. What is FLIP?
10. What is `MotionPath`?

Conclusion

These notes cover beginner to advanced GSAP concepts including: • Core animation methods • Every major animation property • Timelines • ScrollTrigger • React integration • SVG animation • Performance optimization • Professional plugins Mastering GSAP allows developers to create highly interactive and visually impressive user experiences.